

Photo Curation & Cleanup

Product spec — v1.1

"Trasforma 2.000 foto in 60 che rivedrai davvero — e liberati delle altre."

Changelog v1.1 — Il modello di gusto. Riscritta da capo la §5.3: l'apprendimento avviene su **coppie intra-cluster**, non su etichette per giornata (§5.3.2-5.3.4), con **modello globale + adattamento di sessione effimero** (§5.3.5), **esplorazione forzata e soglie asimmetriche permanenti** (§5.3.7), e **autonomia guadagnata sull'accordo misurato** (§5.3.8). Aggiunto lo **spike-2** (§12.1) che falsifica la tesi in un weekend.

Changelog v1.0 — Consolidamento. Niente viene tagliato. Ma il prodotto viene **riorganizzato attorno a due segmenti** e a un'unica intuizione: lo stesso motore serve due bisogni diversi, e **l'audit decide quale**.

- La **selezione** è la funzione centrale; la **cancellazione** ne è la conseguenza (§4), non il punto di partenza.
- L'**Audit** promuove sé stesso da stadio tecnico a **onboarding che configura l'app** (§3).
- Aggiunta la **matrice dogfood** (§11): ciò che l'autrice non può testare su di sé esce dopo, e con un tester del segmento giusto.

1. I due bisogni (e perché sono lo stesso motore)

	Segmento CURATELA	Segmento SPAZIO
Chi	iCloud a pagamento, ottimizzazione attiva	piano gratuito 5 GB / iCloud Foto spento / Android post-Google-Photos
Sul telefono	thumbnail; iOS gestisce già la cache	originali a piena risoluzione
Il dolore	"2.000 foto che non guardo"	"il telefono non scatta più"
Cancellare libera	quota iCloud (€/mese) + rumore	GB veri, subito
Cosa vuole	le 60 buone	i 12 GB
Numerosità	minoranza (paganti)	maggioranza

Nota importante: l'autrice di questo documento appartiene al primo segmento, che è **il più piccolo**. Chi apre l'App Store alle 23 con la memoria piena sta nel secondo. Costruire solo

per sé sarebbe un errore di mercato; costruire solo per gli altri sarebbe un errore di validazione (§11).

La buona notizia: è un solo motore. Stessa scansione, stessi embedding, stessi Processor. Cambia solo *quale operazione l'app mette in cima* — e a deciderlo è l'Audit.

2. Le tre tesi

Tesi A — Foto: decidere per default, non per scelta. L'app propone, l'utente **rivede solo le eccezioni**. Da "scegli 4.000 volte" a "conferma 120 gruppi".

Tesi B — Screenshot: separare il contenuto dal contenitore. Uno screenshot serve a ricordare **una riga di testo**. Estraila, strutturala, e l'immagine diventa cancellabile senza perdita.

Tesi C — Video: conservare i secondi, non i minuti. Non "tenere o cancellare", ma "**quali 8 secondi tenere**".

Le tre tesi sono la stessa tesi: **estrai il valore, butta il contenitore**. Il valore è una foto in un burst, una riga di testo in uno screenshot, otto secondi in un video. E — vedi §4.4 — perfino una **miniatura da 40 KB** in una foto da 3 MB destinata al cestino.

3. ★ L'AUDIT È L'ONBOARDING

L'audit non è lo stadio 0 dell'ingegneria. È **la prima schermata**, ed è il differenziatore del prodotto.

L'app misura — foto vs video vs screenshot, locale vs iCloud, ottimizzato o no, spazio libero — e poi **dice la verità e si configura da sola**:

⚠️ **"Non selezionata" ≠ "cancellabile".** Le **protette** sono foto che non entrano in nessun album ma non si toccano: screenshot con dati ancora vivi, documenti, scontrini, ricevute,

l'unica foto di una persona che compare una volta sola in tutta la libreria. Confondere i due insieme è **il bug più costoso possibile**.

4.2 Non è un ranker. È un selettore.

Il punto tecnico più importante di tutto il documento.

Se prendi le 60 foto col punteggio più alto, ottieni **60 tramonti quasi identici**: tecnicamente perfetti, album inguardabile. Il ranking premia la qualità e **ignora la ridondanza** — esattamente l'errore da non fare quando l'obiettivo è "un album che rivedrai".

Quello che serve è **copertura + qualità**: un ottimo scatto per ogni momento / luogo / persona. È un problema con un nome: **selezione con vincolo di diversità** (massimizzazione submodulare / MMR). In pratica, un greedy da venti righe:

```
python

selezionati = []
while len(selezionati) < quota:
    best = argmax over candidati of (
        score(foto) - λ * max_similarity(foto, selezionati)
    )
    selezionati.append(best)
```

λ è la manopola **varietà ↔ qualità pura**, e sospetto sia l'unico slider che l'utente vorrà davvero toccare.

4.3 Il criterio non è l'estetica

Contro-intuitivo ma probabilmente vero: **le foto che riguardi non sono le più belle. Sono quelle con dentro le persone, e quelle uniche.**

Segnale	Peso	Nota
Volti ricorrenti in libreria	altissimo	le persone care battono qualunque tramonto
Unicità	alta	il soggetto che compare una volta sola > il dodicesimo torii
Comportamento (viste, condivise, editate, favorite)	alta	è già un tuo giudizio: il dato più onesto che esista
Qualità estetica	spareggio	decide <i>quale</i> delle 5 del gruppo, non <i>quale</i> gruppo
Preferenza appresa (§5.3)	media	affina, non decide

4.4 L'archivio fantasma 🧛 — la feature che sblocca la cancellazione

Il motivo per cui non cancelli **non è lo spazio: è la paura del rimpianto**. Ed è risolvibile tecnicamente.

Prima di eliminare, l'app conserva di ogni foto condannata: **miniatura 200px + metadati + testo OCR. 2.000 miniature ≈ 30-40 MB**. Nulla.

Risultato: hai cancellato i GB e **conservi la memoria che quella foto è esistita**. Puoi ancora sfogliare, cercare, sapere che il 16 giugno alle 14:20 avevi fotografato quel menù. L'originale resta comunque nel cestino di sistema per 30 giorni.

È la stessa tesi applicata alla cancellazione stessa: **estrai il valore, butta il contenitore**.
Una brutta foto vale la memoria di essere esistita, non i suoi 2,4 MB.

4.5 L'unità di decisione è il gruppo, non la foto

1.940 conferme singole non le farai mai. Nemmeno mille swipe. Ma:

"12 scatti quasi identici del torii — ne tengo 1, elimino 11." → **un tap** "Il 14 giugno hai scattato 180 foto. Ne tengo 15, elimino 165." → **un tap**

1.940 foto ≈ **120-180 gruppi ≈ una sessione da 20 minuti**. Ogni gruppo mostra **la foto salvata accanto alle scartate**: vedi cosa perdi e cosa tieni al suo posto. È l'unico modo in cui il cervello accetta di lasciare andare.

4.6 Prima della prima cancellazione di massa

Una sola domanda, una volta sola:

"Vuoi scaricare gli originali su Mac / disco esterno prima di procedere?"

Chi dice sì è protetto per sempre. Chi dice no ha scelto consapevolmente. L'app non deve avere opinioni oltre questo.

5. Modulo A — Foto

5.1 Segnali (tutti on-device)

- **Qualità tecnica**: laplacian variance (sfocatura), esposizione, contrasto, rumore
- **Contenuto**: embedding (`VNGenerateImageFeaturePrint`); Android: MobileNet/CLIP-lite via TFLite)
- **Volto**: presenza, numero, occhi aperti, dimensione relativa, **ricorrenza in libreria**
- **Contesto**: EXIF (data, GPS, burst id), (`isFavorite`), editata, condivisa, visualizzazioni
- **Unicità**: distanza dagli altri membri del cluster

5.2 Clustering

- **Near-duplicate / burst**: soglia stretta su embedding + prossimità temporale (< 60s)

- **Evento:** DBSCAN su (tempo, GPS) → "Kyoto, 14 giugno, pomeriggio". È l'unità della quota e della review.

5.3 ★ IL MODELLO CHE IMPARA I TUOI GUSTI

Il cuore intellettuale del prodotto. **Tutto on-device: zero server, nessuna foto lascia il telefono.**

5.3.1 Perché NON i "canoni di bellezza"

I modelli di *attractiveness scoring* pre-addestrati: dataset pieni di bias, output fragili, e non è nemmeno ciò che serve. La domanda non è "questa foto è oggettivamente bella?" ma "la terrei?" — problema molto più facile, e i dati di training li produce l'utente stesso, gratis.

5.3.2 ⚠ La trappola: un giorno non generalizza a un altro giorno

L'idea intuitiva è "addestra sulla giornata: guarda cosa tiene l'utente su 150 foto di Kyoto e applica al resto". Sembra ovvia. **È una trappola**, e va disinnescata prima di scrivere codice.

Addestrando su un giorno, il modello impara **due cose mescolate**:

Cosa impara	Generalizza?	Verdetto
Stile — nitidezza, composizione, come inquadri i volti, odio per le sovraesposte, quanto stretto tagli	✅ sì	oro
Contenuto — "oggi preferisce i dettagli ai panorami, perché è a Kyoto"	❌ no	veleno, se trasportato fuori contesto

Se addestri sul 14 giugno a Kyoto e poi applichi alla cena di compleanno al chiuso, ti propone di cancellare metà della serata. **Il modello ha imparato Kyoto, non te.**

5.3.3 La soluzione: impara dentro il gruppo, non dentro il giorno

▮ Il cluster è un esperimento naturale: controlla il contenuto e isola lo stile.

Quando davanti a **12 scatti quasi identici del torii** scegli il n.3, il contenuto è **costante** in tutti e 12. Non stai dicendo "mi piacciono i torii". Stai dicendo **esattamente e solo**:

▮ "a parità di soggetto, preferisco questa inquadratura, questa nitidezza, questo taglio."

Che è **precisamente e unicamente** il segnale che generalizza.

5.3.4 Conseguenza: non è classificazione, è learning-to-rank

Il dato non è un'etichetta binaria (tengo/butto). Sono **preferenze a coppie**, estratte gratis dalla review:

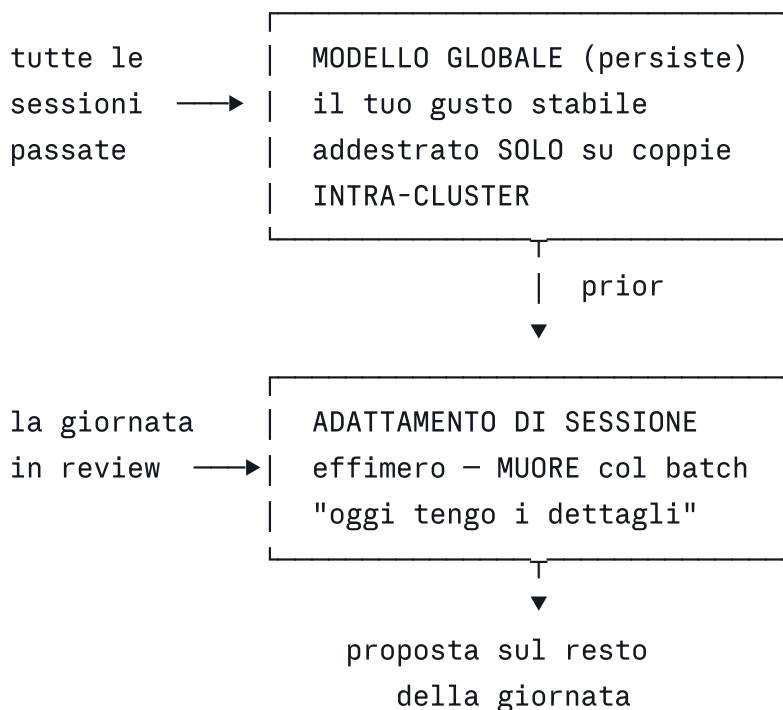
```
cluster torii (12 foto), scelta = #3
⇒ foto_3 > foto_1 , foto_3 > foto_2 , foto_3 > foto_4 , ... = 11 coppie
```

Una giornata di review produce **migliaia di coppie**, senza chiedere all'utente niente in più di quello che stava già facendo.

- **Modello:** Bradley-Terry / RankNet — in pratica una **regressione logistica sulla differenza dei feature vector** delle due foto
- **Efficienza sui dati:** dove servirebbero ~500 etichette binarie, bastano **poche decine di gruppi**
- **Costo:** poche centinaia di feature, addestramento in **millisecondi** su iPhone. Non serve CoreML updatable: basta **Accelerate**.

La difficoltà è tutta nel **cosa** impari, non nel **come**.

5.3.5 Architettura a due livelli



Il **globale** trasporta ciò che generalizza. Il **locale** cattura l'umore del giorno e **poi muore**, senza contaminare il prossimo. È questa separazione che rende sicuro l'apprendimento per giornate.

5.3.6 Il ciclo operativo (batch → impara → applica)

1. L'app raggruppa la giornata (~150 foto → ~18 gruppi)
2. Mostra **5 gruppi** e fa scegliere
3. **Riaddestra** (millisecondi)
4. Propone i restanti **13 gruppi già decisi**
5. L'utente corregge → **ogni correzione è un nuovo label**

6. Il modello globale assorbe le coppie e arriva al giorno dopo più bravo

Active learning: al passo 2 non mostrare 5 gruppi a caso, ma i 5 su cui il modello è **più incerto**. Si impara di più chiedendo dove non si sa, non dove si è già sicuri. Riduce drasticamente le domande necessarie.

Cold start (opzionale, 10 min): ~150 foto in griglia. Ma con l'approccio a coppie **spesso non serve**: la prima review lo addestra da sola.

5.3.7 I due rischi seri

Rischio 1 — La bolla. Il modello impara che tieni i volti → propone volti → tu tieni volti → si rinforza. Dopo sei mesi ha cancellato in silenzio ogni foto di architettura, **e tu non l'hai mai saputo perché non le hai mai viste.** → **Mitigazione: esplorazione forzata.** Una quota del 5-10% delle proposte di "tengo" è casuale (ϵ -greedy). Costa pochissimo e ti salva da te stessa.

Rischio 2 — Il successo che uccide. Il modello diventa bravo → smetti di controllare → un errore sistematico cancella su larga scala. **È il fallimento peggiore possibile, e arriva proprio dopo il successo.** → **Mitigazione: soglie asimmetriche, per sempre.**

La confidenza richiesta per **proporre di cancellare** resta **molto più alta** di quella per proporre di tenere. **Sempre**, anche a modello maturo, anche al 99% di accordo. Sbagliare un *tengo* costa **un tap**. Sbagliare un *cancella* costa **una foto**. La loss deve rispettare questa asimmetria, e **non deve mai calare col tempo**.

5.3.8 L'autonomia si guadagna (ed è revocabile)

Da tutto questo cade fuori un meccanismo di prodotto naturale: **l'accordo misurato è la moneta con cui l'app compra autonomia.**

Settimana 1 — "Confermi questi 18 gruppi?" → accordo **71%** *Settimana 6* — "Confermi questa giornata intera?" → accordo **94%** *Mese 3* — "Ho preparato il tuo viaggio in Giappone: **62 tenute, 1.938 da eliminare.** Vuoi vedere le **12 su cui ero incerta?**"

L'autonomia è **concessa dall'utente sulla base di un numero vero**, ed è **revocabile in qualsiasi momento**. Non è l'app che decide di fidarsi di sé stessa: è l'utente che decide quanto guinzaglio darle, guardando l'accordo misurato.

5.3.9 E comunque: termine, non giudice

Il gusto appreso entra nello score come **un termine tra gli altri** (§4.3), mai come giudice unico. Volti ricorrenti, unicità e segnali comportamentali restano sopra di lui. Il modello raffina la scelta **dentro** il gruppo; non decide **quali** gruppi contano.

5.4 Guardrail non negoziabili

Mai proporre per la cancellazione: **favorite • editate • condivise • unica foto di una persona rara in libreria • foto tecnicamente pessime ma di momenti irripetibili • qualunque asset con una card attiva (§6).**

Un falso positivo costa più di dieci falsi negativi. **La fiducia si perde una volta sola.**

5.5 Spiegabilità

Ogni proposta porta una motivazione in una riga: *"quasi identica a quella che tengo"* · *"sfocata"* · *"occhi chiusi"* · *"foto n. 47 di 52 dello stesso evento"*. È la differenza tra approvare in blocco e disinstallare.

6. Modulo B — Screenshot

6.1 Gli screenshot hanno una scadenza

Categoria	Esempio	Vita utile
Effimero	boarding pass, QR, orario treno, conferma ordine	finisce all'evento → morto con certezza
Riferimento	password wifi, codice sconto, indirizzo, IBAN, tracking	serve finché serve
Wishlist	prodotto, ristorante, libro, film	vive finché non lo consumi
Sapere	ricetta, snippet, thread	archivia il testo, butta l'immagine
Sociale	meme, screenshot di chat, tweet	zero valore dopo 48h
Rumore	schermata home, tap accidentale	morto all'arrivo

Che un boarding pass di **tre settimane** fa sia inutile è l'euristica più potente del prodotto. **Non serve intelligenza: serve una data.**

6.2 Pipeline

screenshot rilevato	→ iOS: PHAssetMediaSubtype.photoScreenshot Android: MediaStore / path Screenshots
OCR on-device detector	→ VNRecognizeTextRequest (IT / EN / JA) → NSDataDetector (date, indirizzi, telefoni, URL, IBAN) VNDetectBarcodesRequest (QR / barcode)
classificazione strutturazione	→ keyword + layout + classificatore piccolo → Card tipizzata
expiresAt	→ dalla data nel testo, o regola per categoria

Detector deterministici prima (coprono ~60% dei casi). Per il resto: **on-device di default** (Foundation Models / Gemini Nano), **cloud solo come toggle opt-in** (LLM con JSON forzato). Mostrare **sempre** il testo OCR grezzo: un errore di parsing dev'essere correggibile, non silenzioso.

6.3 Modello dati

```
json
{
  "id": "uuid",
  "sourceAssetId": "PHAsset local id",
  "type": "booking | receipt | recipe | place | credential | wishlist | snippet",
  "capturedAt": "2026-06-14T09:31:00Z",
  "expiresAt": "2026-06-20T00:00:00Z",
  "confidence": 0.87,
  "rawText": "...",
  "fields": { "title": "...", "datetime": "...", "code": "...", "amount": null,
  "status": "active | expired | archived | exported",
  "sourceKept": true
}
```

`rawText` pesa nulla e rende l'archivio **full-text searchable**. "Cerca tra i tuoi screenshot" è una feature che oggi nessuno fa bene — **e vive del tutto indipendentemente dallo spazio liberato**, il che la rende preziosa in *entrambi* i segmenti.

Vincolo: una card `active` **protegge** il suo asset dalla cancellazione (§5.4).

6.4 Azioni

booking → **Calendario** · place → **Mappe / lista** · recipe → **Note / markdown** · credential → **Keychain** (⚠ default: non processare) · receipt → **CSV** (nota spese di viaggio) · snippet → **file di testo**

Dopo l'azione, **e solo se l'utente lo chiede**: "Ho salvato tutto. Vuoi cancellare lo screenshot?"

6.5 Collezioni

Viaggio Giappone 2026 · Da provare a Roma · Scaduti (47) con bottone *cancella tutto* · **Ricette · Wishlist**

Le collezioni sono il motivo per cui l'app **resta installata dopo la pulizia**. In modalità Curatela, sono forse il prodotto.

7. Modulo C — Video

7.1 Tre vie d'uscita, in ordine di rischio

Azione	Rischio	Spazio	Default
Transcodifica H.264 → HEVC	zero	40-55%	opt-in, mai automatica

Azione	Rischio	Spazio	Default
Trim — proponi il clip	basso	80-95%	produce il clip e basta
Delete	alto	100%	solo esplicito + doppia conferma

Il trim non tocca l'originale. L'app produce un clip; l'originale resta dov'è. Eliminarlo è un **secondo atto**, deliberato. Questo elimina alla radice il rischio peggiore del prodotto.

In **modalità Spazio** i video sono il **primo** modulo: un minuto di 4K/60 $\approx$ 400 MB, e trenta video da viaggio possono valere più di tutta la libreria fotografica.

7.2 Come si "guarda" un video (senza tecnologia nuova)

Un video è una sequenza di frame. I frame passano nella **stessa pipeline delle foto**. Il VideoProcessor non è un secondo motore: è lo stesso motore su una sequenza.

a) Campionamento — `AVAssetImageGenerator`, 1 frame ogni 1-2s a 224px. 2 min $\approx$ 60 embedding. Da qui:

- **near-duplicate cross-mediale** (il clip del Fushimi Inari vs le 14 foto già in libreria)
- **scene detection** (distanza tra embedding consecutivi) → **le scene sono l'unità del trim**
- il **selettore (§4.2) funziona già sui frame**
- **frame migliore** estraibile come foto singola (400 MB → 3 MB)

b) L'audio è il segnale sottovalutato. `SoundAnalysis` (Apple), classificatore built-in, ~300 categorie. Costa poco, discrimina brutalmente:

Audio	Interpretazione	Verdetto
voce umana, risate	qualcuno parla	VETO ASSOLUTO. Non proporre nulla.
musica dal vivo, applausi	concerto / evento	semmai trim generoso
solo vento / ambiente	panoramica mai riguardata	trim proponibile
silenzio + frame scuri, no scene change	video in tasca	proponi eliminazione

Il segnale audio da solo elimina la classe di errore più pericolosa: **toccare il video in cui qualcuno parla.**

c) Etichette zero-shot. MobileCLIP on-device: confronta l'embedding del frame con prompt testuali arbitrari ("*una persona che parla in camera*", "*un video mosso del*

pavimento", "un animale"). Categorie **senza addestrare nulla**. Nuova categoria = nuova frase.

d) Le screen recording sono screenshot lunghi. OCR sui frame → rientrano nel **Modulo B**. Ottimo esempio di perché l'app dev'essere una sola.

7.3 Qualità specifica del video

Shakiness (flusso ottico) · nessun cambio di scena (muro/pavimento/soffitto) · buio persistente · durata anomala (> 3 min a mano ≈ "ho dimenticato di fermare") · **riprodotto zero volte + mai condiviso + girato > 1 anno fa** = tripletta molto forte.

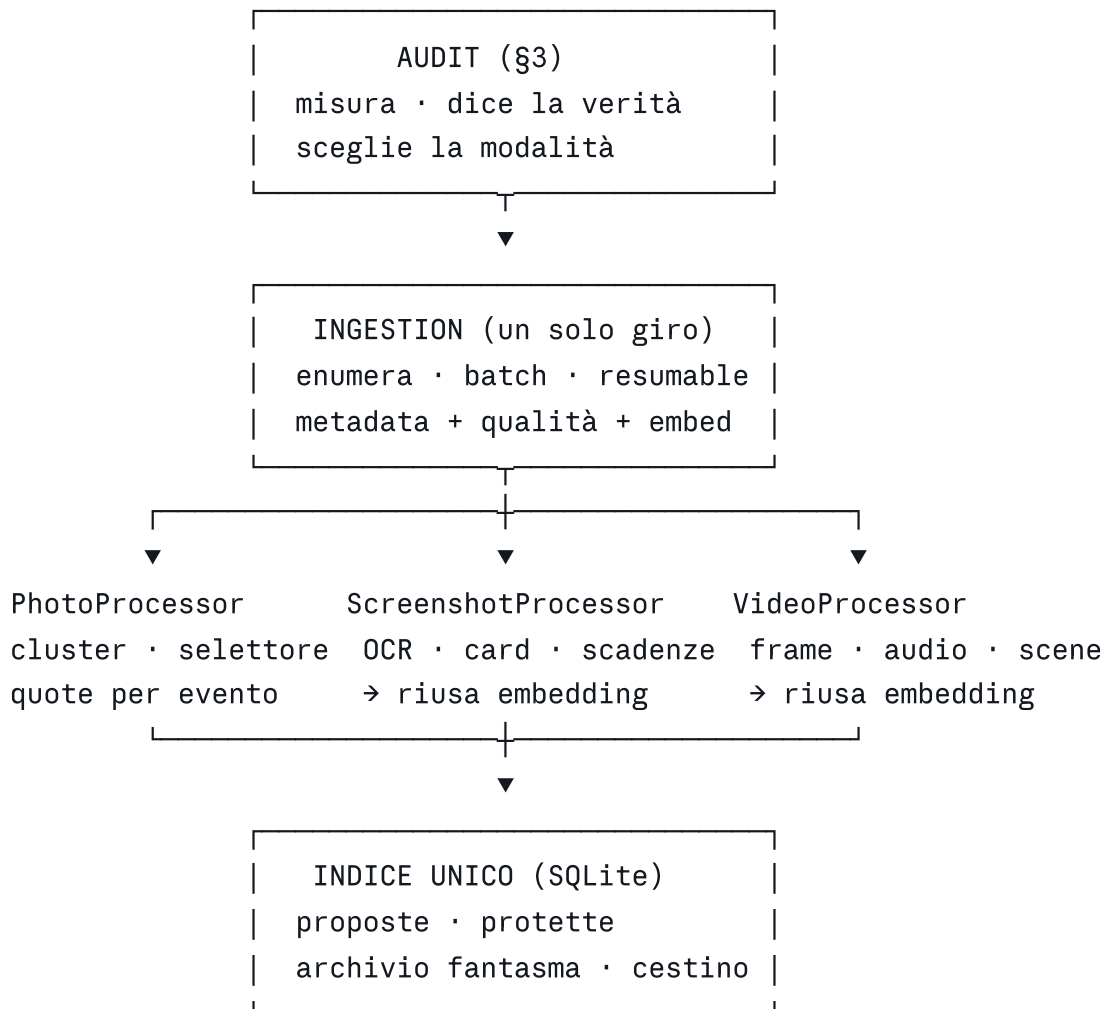
7.4 Costo

Il modulo più caro (decodifica + N embedding + audio). **Sempre in background, su richiesta esplicita** — il che risolve gratis il problema batteria: l'utente sa che sta lavorando, perché gliel'ha chiesto lui. Sotto carica e wifi, progresso visibile, interrompibile, stato persistito. Indicativamente 1-3 s per minuto di video.

8. Architettura

Una app, una scansione, tre Processor.

- La scansione è la parte costosa (I/O, decodifica, embedding, batteria): farla tre volte è spreco puro.
- **Il contatore dev'essere uno.** Spezzato in tre, nessun numero è abbastanza grosso da far agire l'utente.
- **La review dev'essere una.** Tre app = tre giri di dialog di sistema = fatica cognitiva × 3.
- **I segnali si contaminano, in senso buono.** Uno screenshot del biglietto per Kyoto del 14 giugno dà il **nome** al cluster di 200 foto di quel giorno. Un video del Fushimi Inari è near-duplicate di 14 foto già in libreria. Separando, quell'informazione muore.



swift

```
protocol AssetProcessor {  
    func canHandle(_ asset: Asset) -> Bool  
    func process(_ asset: Asset, ctx: PipelineContext) async throws -> [Proposa  
}
```

Il fan-out è **routing dietro un protocollo**, non un `(if)` sparso ovunque. Domani `LivePhotoProcessor` o `DocumentScanProcessor` si aggiungono senza toccare l'ingestion.

9. Il modello d'interazione: console di operazioni

Nessuna azione parte da sola. Mai.

L'app non è un gioco di swipe e non è un maggiordomo che riordina di nascosto. È una **console**: l'utente sceglie *quale operazione* eseguire, l'app la accoda, la esegue **in background**, e torna con una **proposta**.

UTENTE sceglie l'operazione

↓

JOB in coda → esecuzione in BACKGROUND (progresso visibile, interrompibile)

↓

PROPOSTA → l'app non ha modificato nulla. Ha prodotto un elenco.

↓

UTENTE rivede PER GRUPPI (§4.5) → conferma → DOPPIA CONFERMA → cestino

Catalogo

Operazione	Modalità	Cosa fa	Cosa NON fa
Audit	entrambe	dice dove sta lo spazio e la verità su iCloud	nulla, per definizione
Seleziona il meglio	Curatela	quota + diversità → l'album	non cancella
Trova duplicati	entrambe	raggruppa burst/near-dup, propone il migliore	non cancella
Analizza screenshot	entrambe	OCR, card, scadenze	non cancella
Screenshot scaduti	entrambe	elenca le card oltre <code>expiresAt</code>	non cancella
Proponi clip	Spazio	scene detection → intervallo buono	produce il clip e basta
Transcodifica HEVC	Spazio	riduce il peso dei video selezionati	mai automatica; A/B prima
Purge tecnica	Spazio	mosse, scure, schermate morte	non cancella
Cancella il resto	entrambe	il complemento della selezione (§4.1)	mai senza archivio fantasma
Best-of / racconto	Curatela	selezione narrativa dell'evento	non cancella niente

I quattro principi

1. **Niente automatismi.** Ogni operazione parte da un tap.
2. **Prima si salva, poi si cancella.** (§4.1)
3. **Il trim propone il clip. La transcodifica è una scelta.**
4. **La cancellazione avviene per gruppi, con l'archivio fantasma già scritto.**

È più lento da usare e infinitamente più difendibile. E risolve gratis il problema termico/batteria: l'utente sa che l'app sta lavorando, perché gliel'ha chiesto lui.

10. Vincoli di piattaforma

1. **Sandbox:** nessun accesso ai dati di altre app. I **6.53 GB di OldRoll** sono irraggiungibili da qualunque app di terze parti. Le uniche leve sono di sistema (offload / elimina app) o interne all'app stessa. → *Un'app onesta lo dice nell'audit, invece di far finta di niente.*
 2. **iCloud ottimizzato:** cancellare libera **poco** sul dispositivo e **tutto** su iCloud. Avviso in onboarding, non nei settings.
 3. **Nessuna cancellazione silenziosa:** `PHPhotoLibrary.performChanges` → alert di sistema, sempre. **Ma è una conferma per batch:** "600 elementi → 1 tap". È l'unica cosa che *deve* funzionare.
 4. **Su iOS gli album non sono cartelle.** Sono puntatori: un album "Da eliminare" **non libera un byte**.
 5. **Cestino di sistema:** 30 giorni. Lo spazio si libera davvero solo dopo. Va detto, o l'utente cancella 3.000 foto, guarda lo spazio e non è cambiato nulla.
 6. **Memoria:** 40k asset non ci stanno in RAM. Batch + background + stato persistito, **dal giorno 1**. È metà del lavoro di ingegneria seria.
 7. **Trim/transcodifica:** `AVAssetExportSession`, **preservando i metadata** (EXIF, GPS, data di creazione). Un clip che perde la data finisce in fondo alla libreria e l'utente ti odia.
-

11. ★ MATRICE DOGFOOD — cosa puoi validare tu, e cosa no

Il rischio del "teniamo tutto": costruire alla cieca le funzioni che **non puoi usare su di te** — e sono proprio quelle **distruttive**, dove sbagliare costa di più.

Modulo	Segmento	Puoi testarlo tu?	Conseguenza
Selettore / album	Curatela	✅ ogni giorno	prima cosa da costruire
Screenshot / card / ricerca	entrambe	✅ ogni giorno	seconda
Duplicati / burst	entrambe	✅ sì	terza
Cancellazione per gruppi	entrambe	✅ sì (ma libera poco: la validi come <i>curatela</i> , non come <i>spazio</i>)	quarta
Purge tecnica aggressiva	Spazio	⚠️ solo in parte	serve un tester
Video: trim / scene / audio	Spazio	⚠️ poco (poche ore di video?)	serve un tester
Transcodifica	Spazio	❌ no (non libera nulla da te)	serve un tester, obbligatorio
Modalità Spazio nel suo insieme	Spazio	❌ no	serve un tester, obbligatorio

Regola: ciò che non puoi mangiare tu, **esce dopo** — e con un tester vero del segmento "spazio disperato" (telefono da 128 GB, iCloud gratuito, memoria piena). **Un solo tester in quel segmento vale sei settimane di supposizioni.**

12. Lo spike: misurabile, in un weekend

Non serve indovinare se l'algoritmo funziona. **Si misura**, su dati che esistono già.

1. Prendi **una giornata del Giappone** (~150 foto).
2. **Scegli tu, a mano, le tue 15.** ← *ground truth*. Fallo **prima** di scrivere l'algoritmo, o ti autoinganni.
3. L'algoritmo (§4.2) ne sceglie 15.
4. **Metrica: recall@15.**

Recall	Verdetto
$\geq 12/15$	il progetto è vivo → vai avanti
8-11/15	serve il ranker personale → la calibrazione ha senso
$\leq 7/15$	l'approccio è sbagliato → ripensalo prima di investire mesi

Il dataset è già perfetto: 2.000 foto del Giappone, multi-regione, multi-giorno, con dentro tutti i casi difficili — le 200 foto del tempio, i burst dalla bici sulla Shimanami, i cieli del tifone, gli screenshot dei treni. **Se funziona lì, funziona ovunque.**

12.1 Spike-2 — il gusto generalizza davvero? (secondo weekend, opzionale)

Lo spike-1 valida il **selettore**. Il modello di gusto (§5.3) ha bisogno di un suo esperimento, ed è quello che smaschera la trappola di §5.3.2:

1. Addestra sulle **coppie intra-cluster del giorno A** (Kyoto).
2. Testa sul **giorno B, di natura diversa** (una cena al chiuso, non un tempio).
3. **Metrica: accuratezza sulle coppie del giorno B** — data una coppia dello stesso cluster, il modello indovina quale hai tenuto?

Accuratezza su B	Verdetto
$\geq 75\%$	il gusto generalizza → il modello a due livelli funziona
60-74%	generalizza debolmente → serve più contesto o più feature
$\sim 50\%$	sta tirando a caso: hai imparato Kyoto, non l'utente → ripensa le feature

Controllo obbligatorio: ripeti l'esperimento addestrando su etichette binarie *tengo/butto* invece che su coppie intra-cluster. **Se le coppie non battono le binarie sul giorno B, l'intera tesi di §5.3.3 è sbagliata** — ed è meglio saperlo con due giorni di lavoro che con due mesi.

13. Roadmap

Fase	Contenuto	Durata
Fase 0 — stasera	Svuota OldRoll a mano (~6 GB, zero codice) + verifica se ha un <i>clear</i> cache interno	20 min
Spike-1	§12. Ingestion + embedding + clustering + selettore + recall@15 su una giornata	1 weekend

Fase	Contenuto	Durata
Spike-2	§12.1. Coppie intra-cluster → il gusto generalizza al giorno B?	1 weekend
MVP-0	Console + job queue + background + Audit onesto + Selettore → album	3-4 settimane
MVP-1	Cancellazione per gruppi + protette + archivio fantasma + doppia conferma + export originali	2-3 settimane
MVP-1.5	Modello di gusto (§5.3): coppie intra-cluster + due livelli + active learning + soglie asimmetriche + accordo misurato. <i>Si innesta sulla review che già esiste: le etichette arrivano gratis.</i>	2-3 settimane
MVP-2	Screenshot : OCR + card + scadenze + ricerca full-text + collezioni	4-5 settimane
MVP-3	Modalità Spazio : purge tecnica + video (trim, scene, audio) + transcodifica ⚠ <i>richiede tester esterno</i>	4-6 settimane
MVP-4	Autonomia progressiva + best-of / racconto	3-4 settimane
v1.0 store	Onboarding (con gli avvisi iCloud), permessi, edge case, librerie 40k+, privacy policy, empty state	× 2

Fino a "app che uso io e mi risolve il problema": ~2-3 mesi. Fino allo store, con entrambe le modalità: 6-8 mesi part-time.

Ferma dopo MVP-2 e usala per un mese. È l'unico modo per sapere se MVP-3 e MVP-4 servono davvero, o se duplicati + selettore + screenshot bastavano già.

14. Stack

Nativo iOS. PhotoKit + Vision + CoreML + AVFoundation + SoundAnalysis **sono** l'app. In Flutter ci arrivi solo via platform channel, e finiresti a scrivere il 70% della logica in Swift comunque, con in più il costo del ponte. Il cross-platform ha senso quando la logica sta nel dominio; qui sta nel sistema operativo.

Swift · SwiftUI · PhotoKit · Vision · CoreML · AVFoundation · SoundAnalysis · GRDB/SQLite · BackgroundTasks

Android (fase 2, dove vive gran parte del segmento Spazio): Kotlin · Compose · MediaStore · MLKit · TFLite · MediaCodec · Room · WorkManager.

15. Rischi

Rischio	Impatto	Mitigazione
L'utente cancella credendo di liberare spazio, e perde le foto da iCloud	fatale	l' Audit onesto (§3) è la mitigazione, ed è anche la feature
Cancellare qualcosa che contava	fatale (fiducia)	prima salva poi cancella · protette ≠ non selezionate · archivio fantasma · doppia conferma · cestino 30gg
Toccare un video insostituibile	fatale ²	voce umana = veto assoluto · trim non tocca l'originale
Il selettore produce 60 tramonti uguali	alto (il prodotto non serve a niente)	vincolo di diversità (§4.2) + misurato con recall@15
Il modello impara il luogo, non l'utente (§5.3.2)	alto	coppie intra-cluster (§5.3.3) + adattamento di sessione effimero + spike-2 (§12.1)
La bolla: il modello si autoconferma e cancella in silenzio intere categorie	alto e invisibile	esplorazione forzata 5-10% (ϵ -greedy) nelle proposte di "tengo" (§5.3.7)
Il successo che uccide: modello bravo → utente smette di controllare → errore sistematico su larga scala	fatale	soglie asimmetriche permanenti — la confidenza per <i>cancellare</i> resta sempre più alta di quella per <i>tenere</i> , anche a modello maturo (§5.3.7)
Costruire alla cieca la modalità Spazio	alto	matrice dogfood (§11) + tester esterno obbligatorio
Trim che perde metadata (data/GPS)	alto	test espliciti su <code>AVAssetExportSession</code>
Transcodifica che degrada visibilmente	medio	opt-in + A/B sul primo video
OCR debole su italiano/giapponese misti	medio	mostrare sempre il testo grezzo + campi correggibili
Libreria enorme → app lenta/crash	alto	batch + background + persistenza, dal giorno 1

Rischio	Impatto	Mitigazione
Credenziali estratte da screenshot	legale	non processarle di default; Keychain, mai DB in chiaro

16. Landscape

Esistono: Slidebox, Swipewipe, Clever Cleaner, e iOS ha il dedup nativo dalla 16.

Tutti vendono "libera spazio". Con iCloud ottimizzato, tutti stanno mentendo un po' — e non possono ammetterlo senza distruggere il proprio pitch. Lo spazio bianco:

1. **Dire la verità sullo spazio** (§3) — l'unico che può farlo è chi non ha bisogno di mentire
2. **Selezionare con diversità, non rankare** (§4.2) — nessuno lo fa, e per questo tutti producono album inguardabili
3. **L'archivio fantasma** (§4.4) — nessuno lo fa, ed è ciò che rende la cancellazione psicologicamente possibile
4. **Estrarre e collezionare i dati dagli screenshot + ricerca full-text**
5. **Trim invece che delete/keep sui video**
6. **Curatela come prodotto**, non come effetto collaterale della pulizia

Azione gratis, stasera: installane uno per 20 minuti. Annotare cosa ti fa incazzare del loro flusso è il miglior brief di prodotto ottenibile senza scrivere una riga.

17. Domande aperte

1. ~~Dove sta davvero lo spazio?~~ ☒ OldRoll 6.53 GB (irraggiungibile) · Foto 5.12 GB (iCloud ottimizzato)
2. ~~iCloud ottimizzato?~~ ☒ Sì → segmento **Curatela**
3. ~~Serve la cancellazione?~~ ☒ **Sì** → ma **dopo** la selezione (§4.1)
4. **OldRoll ha un clear cache interno?** → 20 minuti stasera
5. **Recall@15 sul Giappone: quanto fa?** ← la domanda che decide se il selettore vive. Un weekend (§12).
6. **Le coppie intra-cluster generalizzano al giorno B?** ← la domanda che decide se il modello di gusto vive. Un weekend (§12.1). **Se le coppie non battono le etichette binarie, la tesi §5.3.3 è sbagliata.**
7. **Chi è il tester del segmento Spazio?** ← senza di lui, MVP-3 è costruito al buio
8. Quanto vale la quota iCloud (€/mese) come motivazione d'acquisto reale?
9. λ (varietà vs qualità): quanto è sensibile? Va esposto o tarato una volta per tutte?
10. Quale **accordo minimo** (%) l'utente pretende prima di concedere autonomia sulla giornata intera? (§5.3.8) — si scopre solo usandola.

La 5, la 6 e la 7 sono quelle che contano. Il resto è ingegneria.